

LAB 3

TESTING & CI/CD FOR ML SYSTEMS

Ensuring Quality and Automating Deployment of ML Applications

Course	DDM501 - AI in Production: From Models to Systems
Session	Session 7
Weight	15% of total grade
Format	Team Lab (3-4 members per team)
Prerequisites	Lab 1 and Lab 2 completed

1. OVERVIEW

1.1. Introduction

In this lab, you will implement comprehensive testing strategies and set up CI/CD pipelines for your movie rating prediction system. Testing ML systems is fundamentally different from testing traditional software - you must test not only code correctness but also data quality, model behavior, and system integration.

You will learn to write different types of tests (unit, integration, data, model), set up automated testing with GitHub Actions, and create deployment pipelines that ensure only high-quality code reaches production.

1.2. Connection to Theory

This lab applies knowledge from the following theory sessions:

- **Session 6:** Testing ML Systems - Testing pyramid, test types, behavioral testing
- **Session 7:** Deployment Strategies & CI/CD - Pipeline design, automation

1.3. Scenario: Quality Assurance for Production

Your movie rating prediction system is about to go live. Before deployment, the team needs to ensure:

- All code changes are automatically tested before merging
- Data quality is validated before training
- Model performance meets minimum thresholds
- API endpoints behave correctly under various conditions
- Deployments are automated and rollback-ready

2. BACKGROUND KNOWLEDGE

2.1. ML Testing Pyramid

The ML Testing Pyramid extends traditional testing to include ML-specific concerns:

Level	Test Type	Examples
Unit	Individual functions/classes	Data transformations, utility functions
Integration	Component interactions	API endpoints, pipeline stages
Data	Data quality & schema	Missing values, distributions, types
Model	Model behavior	Predictions, invariance, performance
System/E2E	Full system behavior	End-to-end workflows, load tests

2.2. Types of ML Tests

Behavioral Testing (CheckList approach)

- **Invariance Tests:** Output shouldn't change for certain input perturbations
- **Directional Tests:** Output should change in expected direction for input changes
- **Minimum Functionality Tests:** Simple cases the model must handle correctly

Data Tests

- **Schema Validation:** Correct data types and structure
- **Distribution Tests:** Data within expected ranges

- **Completeness Tests:** No unexpected missing values

2.3. CI/CD for ML

CI/CD (Continuous Integration/Continuous Deployment) automates testing and deployment:

Stage	Actions
Continuous Integration	Run tests, linting, type checking on every commit/PR
Continuous Delivery	Build artifacts, run integration tests, stage for deployment
Continuous Deployment	Automatically deploy to production after all checks pass

2.4. GitHub Actions Overview

GitHub Actions is a CI/CD platform integrated with GitHub. Key concepts:

- **Workflow:** Automated process defined in YAML files
- **Job:** Set of steps that execute on the same runner
- **Step:** Individual task (run command or action)
- **Action:** Reusable unit of code
- **Runner:** Server that runs workflows

3. HANDS-ON GUIDE

Task 1: Unit Tests for ML Components

1.1. Test Structure

```
tests/
├── __init__.py
├── conftest.py          # Shared fixtures
├── unit/
│   ├── __init__.py
│   ├── test_model.py    # Model loading/prediction tests
│   ├── test_schemas.py # Pydantic schema tests
│   └── test_utils.py    # Utility function tests
├── integration/
│   ├── __init__.py
│   └── test_api.py      # API endpoint tests
├── data/
│   ├── __init__.py
│   └── test_data_quality.py # Data validation tests
└── model/
    ├── __init__.py
    └── test_model_behavior.py # Behavioral tests
```

1.2. Writing Unit Tests

```
# tests/unit/test_model.py
import pytest
from app.model
import MovieRatingModel
```

```

class TestMovieRatingModel:
    """Unit tests for MovieRatingModel class."""
    @pytest.fixture
    def model(self):
        """Fixture to load model once for all tests."""
        return MovieRatingModel()
    def test_model_loads_successfully(self, model):
        """Test that model loads without errors."""
        assert model.is_loaded()
        assert model.model is not None
    def test_predict_returns_float(self, model):
        """Test that predict returns a float value."""
        result = model.predict("196", "242")
        assert isinstance(result, float)
    def test_predict_in_valid_range(self, model):
        """Test that predictions are within 1-5 range."""
        result = model.predict("196", "242")
        assert 1.0 <= result <= 5.0
    def test_predict_batch_returns_list(self, model):
        """Test batch prediction returns correct structure."""
        pairs = [("196", "242"), ("186", "302")]
        results = model.predict_batch(pairs)
        assert isinstance(results, list)
        assert len(results) == len(pairs)

```

1.3. Testing with Fixtures

```

# tests/conftest.py
import pytest
from fastapi.testclient import TestClient
from app.main import app
from app.model import MovieRatingModel
@pytest.fixture(scope="session")
def test_client():
    """Create test client for API tests."""
    return TestClient(app)
@pytest.fixture(scope="session")
def trained_model():
    """Load model once for all tests."""
    return MovieRatingModel()
@pytest.fixture
def sample_prediction_request():
    """Sample valid prediction request."""
    return {"user_id": "196", "movie_id": "242"}

```

```
@pytest.fixture
def sample_batch_request():
    """Sample batch prediction request."""
    return {
        "predictions": [
            {"user_id": "196", "movie_id": "242"},
            {"user_id": "186", "movie_id": "302"},
        ]
    }
```

Task 2: Integration Tests for API (45 minutes)

2.1. API Endpoint Tests

```
# tests/integration/test_api.py
import pytest
from fastapi.testclient
import TestClient

class TestHealthEndpoint:
    """Integration tests for health endpoint."""
    def test_health_returns_200(self, test_client):
        response = test_client.get("/health")
        assert response.status_code == 200
    def test_health_response_structure(self, test_client):
        response = test_client.get("/health")
        data = response.json()
        assert "status" in data
        assert "model_loaded" in data

class TestPredictEndpoint:
    """Integration tests for predict endpoint."""
    def test_predict_valid_request(self, test_client, sample_prediction_request):
        response = test_client.post("/predict", json=sample_prediction_request)
        assert response.status_code == 200
        data = response.json()
        assert "predicted_rating" in data
        assert 1.0 <= data["predicted_rating"] <= 5.0
    def test_predict_missing_user_id(self, test_client):
        response = test_client.post("/predict", json={"movie_id": "242"})
        assert response.status_code == 422
    def test_predict_invalid_json(self, test_client):
        response = test_client.post(
            "/predict",
            content="invalid json",
            headers={"Content-Type": "application/json"}
```

```
)
    assert response.status_code == 422
```

2.2. Testing Error Handling

```
class TestErrorHandling:
    """Tests for API error handling."""
    def test_404_for_unknown_endpoint(self, test_client):
        response = test_client.get("/unknown")
        assert response.status_code == 404
    def test_method_not_allowed(self, test_client):
        response = test_client.get("/predict") # Should be POST
        assert response.status_code == 405
    def test_large_payload_rejected(self, test_client):
        """Test that extremely large payloads are rejected."""
        large_payload = {"user_id": "1" * 10000, "movie_id": "242"}
        response = test_client.post("/predict", json=large_payload) # Should be
        rejected by validation
        assert response.status_code in [400, 422]
```

Task 3: Data Validation Tests

3.1. Custom Data Validators

```
# tests/data/test_data_quality.py
import pytest
import numpy as np
class TestDataQuality:
    """Data quality validation tests."""
    @pytest.fixture
    def sample_ratings(self):
        """Sample ratings data for testing."""
        return [
            {"user_id": "1", "movie_id": "10", "rating": 4.0},
            {"user_id": "1", "movie_id": "20", "rating": 3.5},
            {"user_id": "2", "movie_id": "10", "rating": 5.0},
        ]
    def test_ratings_in_valid_range(self, sample_ratings):
        """Test all ratings are between 1 and 5."""
        for record in sample_ratings:
            assert 1.0 <= record["rating"] <= 5.0
    def test_no_missing_user_ids(self, sample_ratings):
        """Test no missing user IDs."""
        for record in sample_ratings:
            assert record["user_id"] is not None
            assert record["user_id"] != ""
```

```

def test_no_missing_movie_ids(self, sample_ratings):
    """Test no missing movie IDs."""
    for record in sample_ratings:
        assert record["movie_id"] is not None
        assert record["movie_id"] != ""

def test_rating_distribution(self, sample_ratings):
    """Test rating distribution is reasonable."""
    ratings = [r["rating"] for r in sample_ratings]
    mean_rating = np.mean(ratings)
    # Mean should be reasonable (between 2 and 4.5)
    assert 2.0 <= mean_rating <= 4.5

```

3.2. Schema Validation

```

# tests/data/test_schema.py
from pydantic import ValidationError
import pytest
from app.schemas import PredictionRequest, PredictionResponse

class TestSchemaValidation:
    """Schema validation tests."""
    def test_valid_prediction_request(self):
        """Test valid request passes validation."""
        request = PredictionRequest(user_id="196", movie_id="242")
        assert request.user_id == "196"
        assert request.movie_id == "242"

    def test_missing_required_field(self):
        """Test missing required field raises error."""
        with pytest.raises(ValidationError):
            PredictionRequest(user_id="196") # Missing movie_id

    def test_invalid_type(self):
        """Test invalid type raises error."""
        with pytest.raises(ValidationError):
            PredictionRequest(user_id=196, movie_id=242) # Should be strings

```

Task 4: Model Behavioral Tests

4.1. Invariance Tests

```
# tests/model/test_model_behavior.py
import pytest

class TestModelInvariance:
    """Invariance tests - output shouldn't change for certain perturbations."""
    def test_same_input_same_output(self, trained_model):
        """Test that same input always gives same output."""
        result1 = trained_model.predict("196", "242")
        result2 = trained_model.predict("196", "242")
        assert result1 == result2

    def test_order_invariance_for_batch(self, trained_model):
        """Test batch predictions are independent of order."""
        pairs1 = [("196", "242"), ("186", "302")]
        pairs2 = [("186", "302"), ("196", "242")]
        results1 = trained_model.predict_batch(pairs1)
        results2 = trained_model.predict_batch(pairs2)
        # Results should contain same values (possibly different order)
        assert set(results1) == set(results2)
```

4.2. Directional Tests

```
class TestModelDirectional:
    """Directional tests - output should change in expected direction."""
    def test_user_preference_consistency(self, trained_model):
        """Users who rate a movie high should rate similar movies high."""
        # This is a soft test - just checking reasonable behavior
        # User 196 liked movie 242
        pred1 = trained_model.predict("196", "242")    # Predictions should be reasonable
        (not extreme)
        assert 1.0 <= pred1 <= 5.0
```

4.3. Minimum Functionality Tests

```
class TestMinimumFunctionality:
    """Tests for basic functionality that must work."""
    def test_known_user_movie_pair(self, trained_model):
        """Test prediction for known user-movie pair."""
        # User 196 rated movie 242 as 3.0 in the dataset
        prediction = trained_model.predict("196", "242")
        # Prediction should be close to actual (within 1.5)
        assert abs(prediction - 3.0) < 1.5

    def test_handles_new_user(self, trained_model):
        """Test model handles unknown user gracefully."""
        # Should return a reasonable default or raise appropriate error
```



```

    try:
        prediction = trained_model.predict("99999", "242")
        assert 1.0 <= prediction <= 5.0
    except ValueError:
        pass # Acceptable to raise error for unknown user
def test_handles_new_movie(self, trained_model):
    """Test model handles unknown movie gracefully."""
    try:
        prediction = trained_model.predict("196", "99999")
        assert 1.0 <= prediction <= 5.0
    except ValueError:
        pass # Acceptable to raise error for unknown movie

```

Task 5: CI/CD with GitHub Actions

5.1. Basic CI Workflow

```

# .github/workflows/ci.yml
name: CI Pipeline
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.10'
      - name: Cache dependencies
        uses: actions/cache@v3
        with:
          path: ~/.cache/pip
          key: ${{ runner.os }}-pip-${{ hashFiles('requirements.txt') }}
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
          pip install pytest pytest-cov flake8 black mypy

```

```

- name: Run linting
  run: |
    flake8 app/ tests/ --max-line-length=100 black --check app/ tests/
- name: Run type checking
  run: mypy app/ --ignore-missing-imports
- name: Run tests
  run: pytest tests/ -v --cov=app --cov-report=xml
- name: Upload coverage
  uses: codecov/codecov-action@v3
  with:
    file: ./coverage.xml

```

5.2. CD Workflow with Docker

```

# .github/workflows/cd.yml
name: CD Pipeline
on:
  push:
    tags:
      - 'v*'
jobs:
  build-and-push:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3
      - name: Login to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKER_USERNAME }
          password: ${ secrets.DOCKER_PASSWORD }
      - name: Build and push
        uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: |
            ${ secrets.DOCKER_USERNAME }/movie-rating-api:latest
            ${ secrets.DOCKER_USERNAME }/movie-rating-api:${{
github.ref_name }}

```

5.3. Model Validation in CI

```

# .github/workflows/model-validation.yml

```

```

name: Model Validation
on:
  push:
    paths:
      - 'models/**'
      - 'pipeline/**'
jobs:
  validate-model:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.10'
      - name: Install dependencies
        run: pip install -r requirements.txt
      - name: Train model
        run: python scripts/train_model.py
      - name: Validate model performance
        run: |
          python -c "
            from app.model import MovieRatingModel
            model = MovieRatingModel()
            # Add performance validation
            assert model.is_loaded(), 'Model failed to load'
            print('Model validation passed!')
          "

```

4. STARTER CODE TEMPLATE

Clone starter repository:

```
Unzip ddm501-lab3-starter.git
```

Files to complete:

File	TODO Items
tests/unit/test_model.py	Implement unit tests for model class
tests/unit/test_schemas.py	Implement schema validation tests
tests/integration/test_api.py	Implement API endpoint tests
tests/data/test_data_quality.py	Implement data validation tests
tests/model/test_model_behavior.py	Implement behavioral tests
.github/workflows/ci.yml	Create CI pipeline configuration

5. DELIVERABLES & GRADING

5.1. Deliverables

1. **Test Suite:** Comprehensive tests (unit, integration, data, model)

2. **CI/CD Pipeline:** Working GitHub Actions workflows
3. **Code Quality Setup:** Pre-commit hooks and linting configuration
4. **Test Coverage Report:** Minimum 80% code coverage
5. **Documentation:** Testing strategy document and README

5.2. Grading Rubric

Criteria	Weight	Detailed Description
Test Coverage	30%	• Unit tests (10%) • Integration tests (8%) • Data tests (6%) • Model behavioral tests (6%)
CI/CD Pipeline	30%	• CI workflow works (12%) • All checks pass (10%) • CD workflow configured (8%)
Code Quality	20%	• Pre-commit hooks (8%) • Linting passes (6%) • Type hints (6%)
Documentation	20%	• Testing strategy doc (10%) • README updated (5%) • Coverage report (5%)

5.3. Submission

- **Deadline:** 1 week after the lab session
- **Format:** GitHub repository link with passing CI badge
- **Required:** Screenshots of passing CI workflows